



Smarthus – Innbruddsalarmsystem

Prosjektoppgave i ING1507 Datamaskinarkitektur

Kandidat: Oskar Hellebust-Nygaard

Kandidat: Theofor Theiste Haugland

Cyberingeniørskolen – Jørstadmoen

April 2026

Innhold

1	Introduksjon	2
2	Prosjektide	2
2.1	Utfordring	2
2.2	Løsning og virkemåte	2
3	Metode	3
3.1	Utstyr	3
4	Teori	4
4.1	ATmega32-mikrokontroller	4
4.2	LCD HD44780 – 4-bits modus	4
4.3	Touch-sensor – DFRobot V2	5
4.4	Limit switch – vindussensor	5
4.5	Sharp GP2Y0A21 IR-avstandssensor	5
4.6	MG90S servo og Fast PWM	6
4.7	Passiv buzzer og Timer1 CTC	6
4.8	I ² C / TWI-kommunikasjon	6
4.9	Timer0 CTC og sleep mode	7
5	Kobling	7
5.1	Fysisk plassering i 3D-printet hus	7
5.2	Master	8
5.3	Slave 1	8
5.4	Slave 2	9
5.5	I ² C-buss	10
6	Kodeprinsipper	10
6.1	Master – <code>main.c</code>	10
6.2	Slave 1 – <code>main.c</code>	11
6.3	Slave 2 – <code>main.c</code>	13
6.4	I ² C – <code>i2c.h</code>	13
7	Feilsøking og problemer underveis	14
7.1	LCD viste søppeltegn / ingenting	14
7.2	LCD startet på nytt under drift	14
7.3	I ² C protokollkonflikt	14
7.4	IR-sensor konstant trigger	15
7.5	Servo-stalling	15
7.6	Limit switch contact bounce	15
8	Diskusjon	15
8.1	Systemvirkemåte	15
8.2	Interrupt-drevet arkitektur	15
8.3	Begrensninger og mulige forbedringer	16
9	Konklusjon	16
	Referanser	16
A	Vedlegg – Komplette kildekode	17

1 Introduksjon

Prosjektet er et smarthus-alarmsystem med tre ATmega32 som kommuniserer over en felles I²C-buss. Systemet er "montert" inne i et 3D-printet hus og viser praktisk bruk av sentrale emner fra ING1507: digital input/output, external interrupts, timer/counter, PWM, ADC og serial kommunikasjon.

Alarmen har tre tilstander: **DISARMED**, **ARMED** og **ALARM**. Systemet er delt i tre klare roller:

- **Master** – styring, overvåkning og tilstandshåndtering via LCD og touch-sensor.
- **Slave 1** – sensorer: limit switch ved vinduet og IR-avstandssensor på gulvet.
- **Slave 2** – reaksjon: buzzer-sirene og LED-varsling.

Komplett kode til alle tre mikrokontrollere ligger på GitHub: <https://github.com/Oskarhn/Smarthus>

Video av systemets funksjon finnes på YouTube: https://youtu.be/qz9i_bLsLKA

2 Prosjektide

2.1 Utfordring

En fungerende innbruddsalarm krever live overvåkning av flere sensorer, koordinert kommunikasjon mellom uavhengige enheter og god ressurs håndtering på Atmegaene fordi de har begrenset ytelse. Systemet må reagere umiddelbart på sensorene uten blokkerende loops.

2.2 Løsning og virkemåte

Systemet er bygget opp etter en master-slave-arkitektur med tre ATmega32:

- **Master (styring):** Har systemtilstanden og viser den på LCD. Berøring av touch-sensoren veksler mellom ARMED og DISARMED. Master poller slave 1 hvert 500 ms for å oppdage inntrengning og sender kommandoer til begge slavene ved tilstandsending.
- **Slave 1 (sensorer):** Overvåker vindu via limit switch og rommet i 1.etg via IR-sensor. Rapporterer alarm til master og aktiverer servoen som fysisk blokkerer inngangsdøren.
- **Slave 2 (reaksjon):** Reagerer på CMD_ALARM fra master med lyd fra aktiv buzzer og LED-blink.

Alarmsyklusen:

1. Systemet starter i DISARMED. LCD viser beskjed om å armere.
2. Touch sender CMD_ARMED til begge slaver og veksler til ARMED. Slave 2 gul LED tennes som en armert-indikator.
3. I ARMED poller master slave 1 hvert 500 ms. Dersom limit switch eller IR-sensor trigger, rapporterer slave 1 STATUS_DOOR_OPEN.
4. Master sender CMD_ALARM: slave 1 aktiverer servoen som blokkerer døren og blinker rød LED; slave 2 starter buzzersirene og blinker rød LED. LCD viser !! ALARM !!.
5. Ny touch sender CMD_DISARMED og tilbakestiller alle tre.

3 Metode

Under Utviklingen testet vi en komponent av gangen for å sikre at hver del fungerte som det skal. Rekkefølgen var:

1. Touch-sensor med enkel LED-togling (master, ingen LCD)
2. Touch + LCD på master
3. Limit switch + IR-sensor + servo på slave 1 (uten I²C)
4. Buzzer + LED-er på slave 2 (uten I²C)
5. I²C-integrasjon mellom alle tre kort
6. Optimalisering: interrupt-drevet logikk, tomme while-løkker, sleep mode

Inspirasjon og teori ble hentet fra ING1507-leksjonene [1] og datasheet for hvert komponent.

3.1 Utstyr

Master

- 1× ATmega32 (eget breadboard)
- 1× DFRobot V2 touch-sensor (3-pinnere)
- 1× 16×2 LCD-modul (HD44780-kompatibel)
- 1× Potensiometer (LCD-kontrast)
- 100 μ F kondensator (spenningsstabilisering)
- Breadboard, ledninger

Slave 1

- 1× ATmega32 (eget breadboard)
- 1× Limit switch NO (vindussensor, 1. etasje)
- 1× Sharp GP2Y0A21YK0F IR-avstandssensor (gulvsensor, 1. etasje)
- 1× MG90S mikro-servo (180°, ved inngangsdøren)
- 1× Hvit LED (alarmindikator)
- 1× 33 μ F kondensator (IR-sensorfilter)
- Breadboard, ledninger

Slave 2

- 1× ATmega32 (eget breadboard)
- 1× Aktiv buzzer (2. etasje)
- 1× Rød LED
- 1× Gul LED
- Breadboard, ledninger

4 Teori

4.1 ATmega32-mikrokontroller

ATmega32 er en 8-bits RISC-mikrokontroller fra Microchip med 32 KB flash-minne, 2 KB SRAM og 1 KB EEPROM [2]. Den kjøres ved 1 MHz intern oscillator. Relevante periferienheter:

- **GPIO:** Fire 8-bits porter (PA–PD), retning konfigureres via DDRx.
- **Timer0:** 8-bits timer/counter, CTC-modus, brukt til 1 ms systemtakt.
- **Timer1:** 16-bits timer/counter, Fast PWM og CTC-modus, brukt til servo og buzzer.
- **ADC:** 10-bits analog-til-digital-omformer med 8 kanaler (PA0–PA7).
- **TWI:** Two Wire Interface (I²C-kompatibel) på PC0 (SCL) og PC1 (SDA).
- **INT0/INT1:** External interrupt, konfigurerbar på rising eller falling edge.

Alle tre kort konfigureres likt i `main()`, med interrupt-logikk i dedikerte ISR-er (Interrupt Service Routines). Sleep mode brukes mellom interrupts:

```
1 set_sleep_mode(SLEEP_MODE_IDLE);  
2 while (1) { sleep_mode(); }
```

Listing 1: Sleep-loop – alle tre kort

4.2 LCD HD44780 – 4-bits modus

LCD-modulen styres av HD44780-kontrolleren med to typer minne: *DDRAM* (lagrer tegnene som vises) og *CGROM* (innebygde tegnsett) [5]. I 4-bits modus kobles bare DB4–DB7, noe som sparer fire pinner sammenlignet med 8-bits modus.

I 4-bits modus sendes en byte som to nibbler: øvre nibble (bit 7–4) sendes først, deretter nedre nibble (bit 3–0), med en Enable-puls etter hver nibble. Kontrollpinnene:

- **RS** (Register Select): RS=0 kommando, RS=1 data til DDRAM.
- **E** (Enable): LCD leser databussen på falling edge av E-pulsen.
- **RW** koblet permanent til GND – kun skriving brukes.

Nibble-overføringen skriver til de fire laveste bitene av PORTB:

```
1 PORTB = (PORTB & 0xF0) | (nibble & 0x0F);
```

Listing 2: Nibble-skriving til LCD (lcd.h)

Tabell 1 viser de viktigste kommandoene fra initialiseringen.

Tabell 1: LCD-kommandoer brukt i oppgaven

Kommando	Hex	Beskrivelse
Function Set	0x28	4-bit modus, 2 linjer, 5×8 font
Display ON	0x0C	Display på, marker av, blinking av
Clear display	0x01	Sletter DDRAM, markør til home
Entry mode	0x06	Marker beveger seg til høyre
Set DDRAM linje 1	0x80 + col	Setter markerposisjon linje 1
Set DDRAM linje 2	0xC0 + col	Setter markerposisjon linje 2

Oppstartssekvensen sender nibble 0x03 tre ganger (tvinger kjent 8-bits tilstand), deretter nibble 0x02 (bytter til 4-bits). Dette er 4-bits oppstartssekvensen fra HD44780-datasheeten.

4.3 Touch-sensor – DFRobot V2

DFRobot V2 er en kapasitiv touch-sensor med tre pinner (VCC, GND, SIG). Signal- pinnen går HIGH ved berøring og er koblet til PD2 (INT0) på master. Rising-edge interrupt (ISC01=1, ISC00=1) trigges umiddelbart ved berøring uten polling.

Interrupt-konfigurasjonen i main():

```
1 MCUCR |= (1 << ISC01) | (1 << ISC00); /* Rising edge */
2 GICR  |= (1 << INT0);                /* Enable INT0 */
```

Listing 3: INT0 rising-edge interrupt konfigurasjon (Master/main.c)

4.4 Limit switch – vindussensor

En limit switch med NO-kontakt (Normally Open) er montert ved vinduet i 1. etasje. Bryteren aktiveres når vinduet åpnes, noe som gir innbruddsdeteksjon for noen som forsøker å åpne vinduet eller klatre inn.

Med intern pull-up på PD2 er signalet normalt HIGH. Når bryteren aktiveres (vinduet åpnes) faller signalet til GND og genererer en rising-edge INT0. Software debounce på 200 ms forhindrer contact bounce:

```
1 ISR(INT0_vect)
2 {
3     if (!dor_timer) { dor_apnet = 1; dor_timer = 200; }
4 }
```

Listing 4: INT0 ISR – vindussensor med debounce (Slave1/main.c:42)

4.5 Sharp GP2Y0A21 IR-avstandssensor

Sharp GP2Y0A21 er en analog IR-avstandssensor med måleområde 10–80 cm [3]. Sensoren sender ut infrarødt lys og måler det reflekterte signalet via en posisjonsfølsom detektor (PSD). Utgangsspenningen er omvendt proporsjonal med avstand: høy spenning betyr kort avstand, lav spenning betyr lang avstand.

Sensoren er plassert i 1. etasje og peker ned mot gulvet. Den detekterer noen som beveger seg inn i rommet. ADC-verdien (10-bit, 0–1023) er høy nær sensoren. En terskelverdi på IR_TERSKEL = 500 tilsvarer omtrent 20 cm. Sensor- databladet krever en 10–33 μ F avkoblingskondensator direkte på VCC/GND for stabil drift ved de kortvarige IR-LED-pulsene.

ADC startes i kode hvert 40 ms via Timer0 ISR:

```
1 if (ir_armed && !ir_cooldown && ++ir_tick >= 40)
2     { ir_tick = 0; ADCSRA |= (1 << ADSC); }
```

Listing 5: ADC-start hvert 40ms i Timer0 ISR (Slave1/main.c)

Når konverteringen er ferdig kjøres ADC_vect interrupt automatisk:

```
1 ISR(ADC_vect)
2 {
3     if (ADC > IR_TERSKEL) ir_trigger = 1;
4 }
```

Listing 6: ADC interrupt – IR-terskel (Slave1/main.c:48)

4.6 MG90S servo og Fast PWM

MG90S-servoen er plassert ved inngangsdøren og brukes til å fysisk blokkere døren når alarmen utløses. En servo-motor styres med Pulse Width Modulation (PWM): perioden er fast (20 ms), og pulsbredden bestemmer vinkelposisjonen [4]:

- $500 \mu s \approx 0^\circ$
- $1500 \mu s \approx 90^\circ$ (nøytral)
- $2400 \mu s \approx 180^\circ$

Timer1 konfigureres i Fast PWM-modus med ICR1 som TOP (WGM13+WGM12+WGM11, COM1A1, prescaler 8). Ved 1 MHz og prescaler 8 er ett tikk $8 \mu s$, og ICR1=2499 gir $2500 \times 8 \mu s = 20 \text{ ms}$ periode. OCR1A-verdiene:

$$\text{OCR1A} = \frac{\text{Pulsbredde} [\mu s]}{8 \mu s / \text{tikk}}$$

$$\text{SERVO_LUKKET} = 188 \text{ (} 1504 \mu s \text{)}, \quad \text{SERVO_AAPEN} = 300 \text{ (} 2400 \mu s \text{)}$$

Servoen styres ved å sette OCR1A direkte i Timer0 ISR-en:

```
1 OCR1A = SERVO_AAPEN; /* Alarm: blokker door */
2 OCR1A = SERVO_LUKKET; /* Normal: noeytral posisjon */
```

Listing 7: Servo-posisjon settes ved alarm (Slave1/main.c)

4.7 Passiv buzzer og Timer1 CTC

Buzzeren krever et oscillerende signal. Timer1 konfigureres i CTC-modus 12 (ICR1 som TOP) med COM1A0 som toggler OC1A (PD5) ved compare match og prescaler 8. Tonefrekvensen er:

$$f = \frac{f_{\text{CPU}}}{2 \cdot N_{ps} \cdot (\text{ICR1} + 1)} = \frac{62500}{\text{ICR1} + 1}$$

Melodien spilles av fra en tabell med {ICR1, lengde}-par (D5, E5, G5, A5, pause) som oppdateres hvert 50 ms i Timer0 ISR-en:

```
1 if (buzzer_paa && ++siren_tick >= 50)
2 { siren_tick = 0; Buzzer_Melody_Update(); }
```

Listing 8: Melodioppdatering hvert 50ms (Slave2/main.c)

4.8 I²C / TWI-kommunikasjon

I²C er en seriell to-leder-buss (SCL og SDA) som støtter en master og opptil 127 slaver [1]. ATmega32s TWI-hardware implementerer I²C-protokollen. Klokkefrekvensen beregnes med:

$$\text{TWBR} = \frac{f_{\text{CPU}}}{2 \cdot f_{\text{SCL}}} - 8 = \frac{1 \text{ MHz}}{2 \cdot 50 \text{ kHz}} - 8 = 2$$

Med TWBR=2 og prescaler=1 oppnås 50 kHz SCL. Slave-adresser er 0x10 (slave 1) og 0x20 (slave 2). Datafloaten bruker fire kommunikasjonsmodi:

- **MT (Master Transmit):** START → SLA+W → CMD-byte → STOP
- **MR (Master Receive):** START → SLA+R → DATA+NACK → STOP

- **SR (Slave Receive):** Reagerer på status codes 0x60 og 0x80 i TWSR
- **ST (Slave Transmit):** Reagerer på 0xA8, sender `slave_status`

Master sender kommandoer til begge slaver ved tilstandsending:

```
1 static void send_to_slaves(uint8_t cmd)
2 {
3     I2C_Master_Send(SLAVE1_ADDR, cmd);
4     _delay_ms(5);
5     I2C_Master_Send(SLAVE2_ADDR, cmd);
6 }
```

Listing 9: `send_to_slaves()` – broadcast til begge slaver (Master/main.c:39)

Slave-polling er non-blocking: `I2C_Slave_Poll()` returnerer umiddelbart 0 dersom ingen I²C-aktivitet er registrert, slik at ISR-en ikke blokkeres.

4.9 Timer0 CTC og sleep mode

Timer0 konfigureres i CTC-modus med `OCR0=124` og `prescaler=8`:

$$f_{\text{ISR}} = \frac{1 \text{ MHz}}{8 \cdot (124 + 1)} = 1000 \text{ Hz} \Rightarrow \text{ISR hvert 1 ms}$$

All tidsstyrt logikk (debounce, polling-intervaller, LED-blink, alarm-timeout) drives av en global `ms`-teller i denne ISR-en uten bruk av `_delay_ms()`. Tidsintervaller sjekkes med delta-metoden:

```
1 if ((uint16_t)(ms - last_poll) >= 500) { last_poll = ms; /* handling */ }
```

Listing 10: Tidsintervall uten blokkering – delta-metoden

Denne metoden er robust ved overflow av `ms` (`uint16_t` wrapper rundt etter 65535).

`SLEEP_MODE_IDLE` lar CPU-en hvile mellom interrupts. I Idle-modus stopper CPU-kjernen, men Timer0, Timer1, TWI, ADC og INT0 kjører videre. Aktiv CPU-tid reduseres fra $\approx 100\%$ til $\approx 10\text{--}20\%$.

5 Kobling

5.1 Fysisk plassering i 3D-printet hus

Komponentene er plassert i et 3D-printet to-etasjes hus:

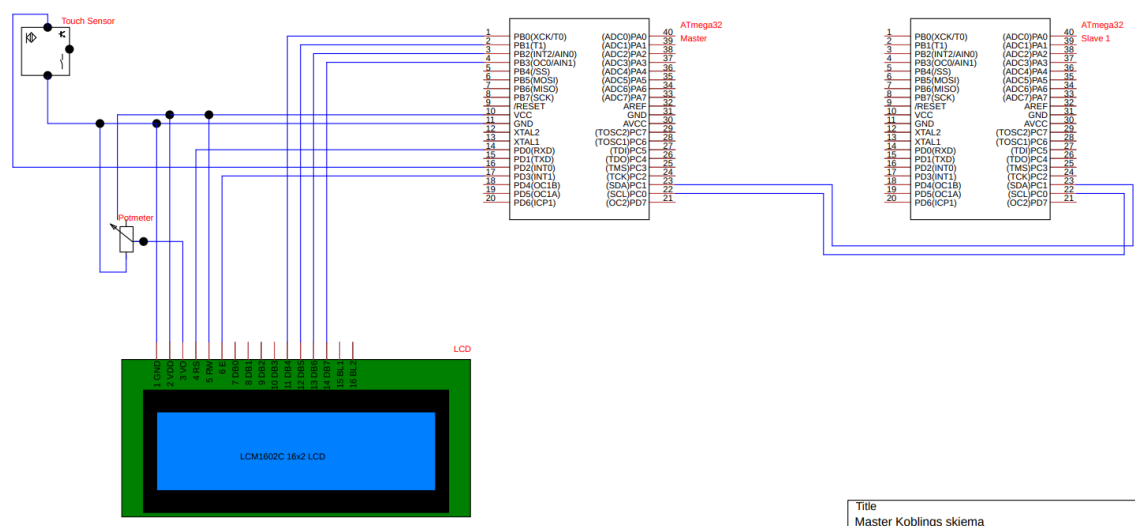
- **2. etasje:** LCD-skjermen er montert i veggen ut mot husets framside, synlig gjennom en åpning. LED-ene stikker ut rett under LCD. Buzzer er plassert oppe i 2. etasje. Master- og slave 2 har hvert sitt lille breadboard i 2. etasje.
- **1. etasje:** IR-sensoren (PIR) peker ned mot gulvet og detekterer noen som beveger seg inn i rommet. Limit switch er montert ved vinduet og trigges hvis vinduet åpnes eller noen klatrer inn. Servoen er plassert ved inngangsdøren og blokkerer den fysisk ved alarm. Slave 1 har eget breadboard i 1. etasje.

5.2 Master

Tabell 2: Pinnekobling – Master

ATmega32-pin	Retning	Komponent	Funksjon
PD2 (INT0)	Inn	Touch-sensor	Touch interrupt
PD0	Ut	LCD – RS	Register Select
PD3	Ut	LCD – E	Enable-puls
PB0	Ut	LCD – D4	Databit 4
PB1	Ut	LCD – D5	Databit 5
PB2	Ut	LCD – D6	Databit 6
PB3	Ut	LCD – D7	Databit 7
PC0 (SCL)	I/U	I ² C-buss	Clock (til slave 1 og 2)
PC1 (SDA)	I/U	I ² C-buss	Data (til slave 1 og 2)

LCD RW er koblet permanent til GND. LCD kontrast (V0) styres av et potensiometer. En 100 μ F kondensator mellom VCC og GND nær ATmega32 stabiliserer spenningen.



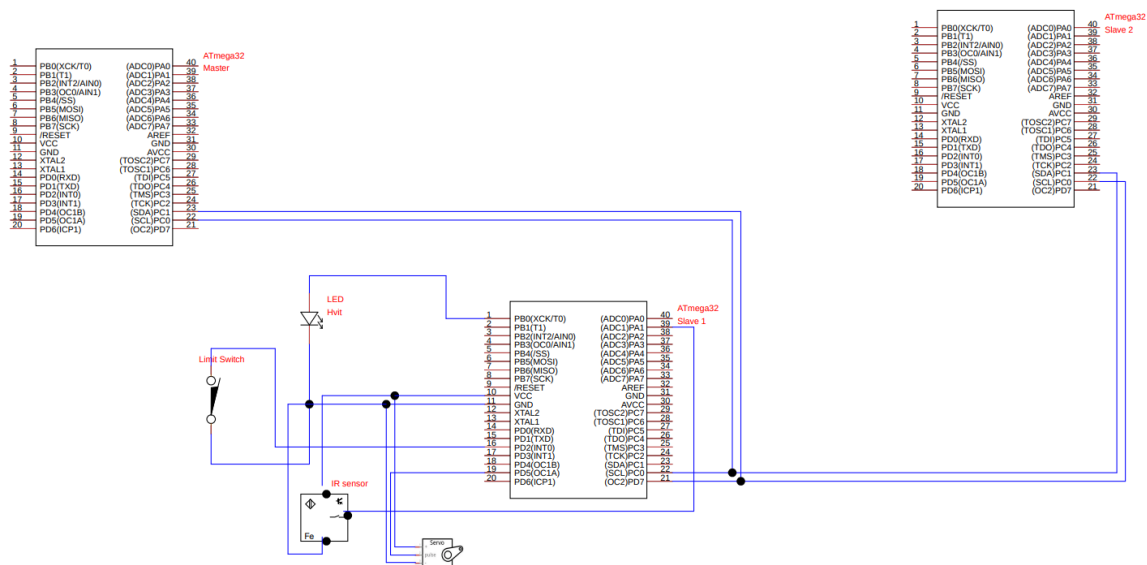
Figur 1: Digitalt koblingsskjema – Master

5.3 Slave 1

Tabell 3: Pinnekobling – Slave 1

ATmega32-pin	Retning	Komponent	Funksjon
PB0	Ut	Rød LED	Alarmindikator
PD2 (INT0)	Inn	Limit switch NO (vindu)	Vindussensor (intern pull-up)
PA1 (ADC1)	Inn	Sharp GP2Y0A21	Avstandsmåling, gulv (analog)
PD5 (OC1A)	Ut	MG90S servo (dør)	Dørblokker (PWM)
PC0 (SCL)	I/U	I ² C-buss	Clock
PC1 (SDA)	I/U	I ² C-buss	Data

AVCC (pin 32) er koblet til VCC via 100 nF kondensator. IR-sensoren har 33 μ F kondensator mellom VCC og GND direkte på sensoren.

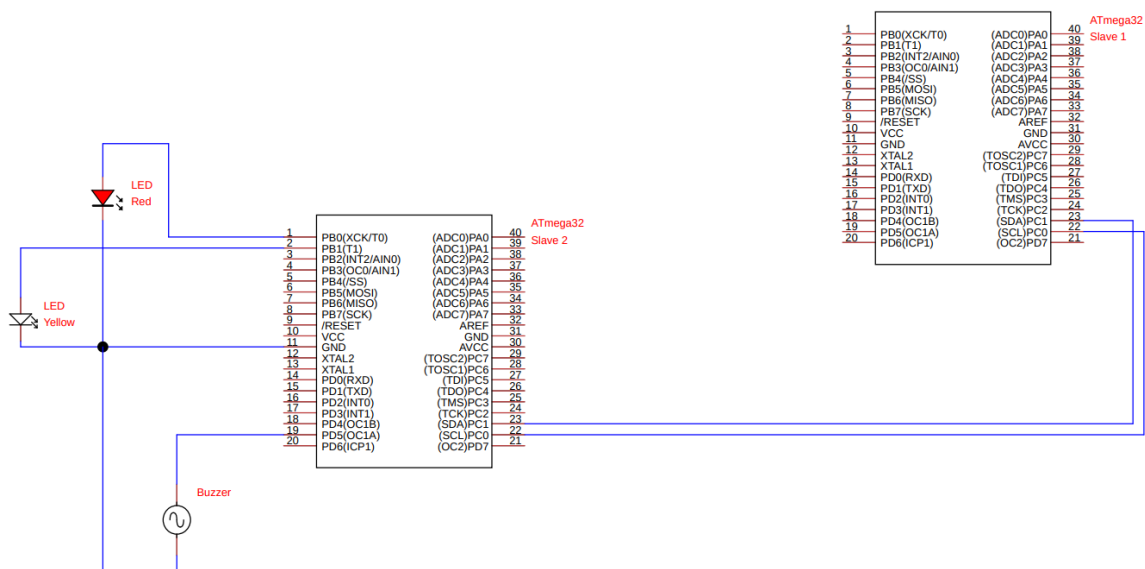


Figur 2: Digitalt koblingsskjema – Slave 1 (sensorer)

5.4 Slave 2

Tabell 4: Pinnekobling – Slave 2

ATmega32-pin	Retning	Komponent	Funksjon
PB0	Ut	Rød LED	Alarmindikator (blinker)
PB1	Ut	Gul LED	Armed-indikator (konstant)
PD5 (OC1A)	Ut	Passiv buzzer	Sirene (PWM)
PC0 (SCL)	I/U	I ² C-buss	Clock
PC1 (SDA)	I/U	I ² C-buss	Data



Figur 3: Digitalt koblingsskjema – Slave 2 (reaksjon)

5.5 I²C-buss

PC0 (SCL) og PC1 (SDA) fra alle tre kort kobles sammen på en felles linje. To 4,7 k Ω pull-up-motstander til VCC monteres på bussen (en per linje). I tillegg aktiveres interne pull-ups på PORTC i koden:

```
1 PORTC |= (1 << PC0) | (1 << PC1);
```

Listing 11: Interne pull-ups på I2C-linjene

6 Kodeprinsipper

All logikk kjøres i ISR-er (Interrupt Service Routines). `while(1)`-løkken på alle tre kort inneholder kun `sleep_mode()` – CPU-en sover til neste interrupt. Dette gir event-driven oppførsel med lite energibruk.

Tre typer interrupts brukes:

- **External interrupt (INT0):** Triggres av touch-sensor (master) eller limit switch (slave 1) på rising edge.
- **Timer0 COMP interrupt:** 1 ms systemtakt på alle tre kort. All tidsstyrt logikk kjøres her.
- **ADC_vect interrupt:** Triggres automatisk når ADC-konvertering er ferdig (kun slave 1).

6.1 Master – `main.c`

Master initialiserer INT0, Timer0, LCD og I²C i `main()`, viser oppstartsskjerm i 1,5 sek, og går deretter i sleep-loop.

INT0 ISR håndterer touch-interrupt med software debounce:

```
1 ISR(INT0_vect)
2 {
3     if (!touch_debounce) { touch_flag = 1; touch_debounce = 50; }
4 }
```

Listing 12: INT0 ISR – touch med debounce (Master/main.c:34)

`touch_flag` settes kun hvis `touch_debounce` er null. Telleren dekrementeres med 1 ms i Timer0 ISR, slik at nye berøringer ignoreres de neste 50 ms – effektiv software debounce.

Timer0 COMP ISR driver hele master-logikken hvert 1 ms:

```
1 ISR(TIMER0_COMP_vect)
2 {
3     static uint8_t  system_state = STATE_DISARMED;
4     static uint16_t last_poll    = 0;
5
6     ms++;
7     if (touch_debounce) touch_debounce--;
8
9     /* Handle touch event */
10    if (touch_flag) {
11        touch_flag = 0;
12        switch (system_state) {
13            case STATE_DISARMED:
14                system_state = STATE_ARMED;
15                send_to_slaves(CMD_ARMED);
16                break;
```

```

17         default: /* ARMED or ALARM */
18             system_state = STATE_DISARMED;
19             send_to_slaves(CMD_DISARMED);
20             break;
21     }
22     lcd_update(system_state);
23 }
24
25 /* Poll slave 1 every 500ms when armed */
26 if (system_state == STATE_ARMED &&
27     (uint16_t)(ms - last_poll) >= 500)
28 {
29     last_poll = ms;
30     if (I2C_Master_Read(SLAVE1_ADDR) == STATUS_DOOR_OPEN) {
31         system_state = STATE_ALARM;
32         send_to_slaves(CMD_ALARM);
33         lcd_update(system_state);
34     }
35 }
36 }

```

Listing 13: Timer0 COMP ISR – state machine og polling (Master/main.c:72)

`system_state` er en `static`-lokal variabel i ISR-en og beholdes mellom kall. LCD oppdateres kun ved tilstandsendring, aldri hvert ms. Polling skjer via tidsdelta-sjekk (`ms - last_poll`) uten å nullstille `ms`-telleren – robust ved overflow.

`lcd_update()` bruker `switch/case` for å skrive korrekt melding:

```

1 case STATE_ARMED:
2     LCD_String_xy(0, 0, "    ARMERT    ");
3     LCD_String_xy(1, 0, "    HUS: OK    ");
4     break;
5 case STATE_ALARM:
6     LCD_String_xy(0, 0, "    !! ALARM !!    ");
7     LCD_String_xy(1, 0, "    HUS: IKKE OK    ");
8     break;

```

Listing 14: `lcd_update()` – tilstandsbasert LCD (Master/main.c:47)

6.2 Slave 1 – `main.c`

Slave 1 er sensor-noden og bruker tre ISR-er som samarbeider:

INT0 ISR registrerer vindussensor-aktivisering:

```

1 ISR(INT0_vect)
2 {
3     if (!dor_timer) { dor_apnet = 1; dor_timer = 200; }
4 }

```

Listing 15: INT0 ISR – vindussensor debounce (Slave1/main.c:42)

ADC_vect ISR kjøres automatisk når ADC-konvertering er ferdig:

```

1 ISR(ADC_vect)
2 {
3     if (ADC > IR_TERSKEL) ir_trigger = 1;
4 }

```

Listing 16: ADC interrupt – IR threshold (Slave1/main.c:48)

IR_TERSKEL = 500 tilsvarer ca. 20 cm avstand. ADC-registeret leses umiddelbart når interrupt kjøres – ingen ventetid.

Timer0 COMP ISR samordner all logikk hvert 1 ms:

```
1 ISR(TIMER0_COMP_vect)
2 {
3     static uint8_t ir_tick      = 0;
4     static uint8_t system_state = STATE_DISARMED;
5     /* ... flere statiske variable ... */
6
7     ms++;
8     if (dor_timer) dor_timer--; /* debounce counter */
9     if (ir_cooldown) ir_cooldown--; /* cooldown etter alarm reset */
10
11     /* Start ADC hver 40ms nar armed */
12     if (ir_armed && !ir_cooldown && ++ir_tick >= 40)
13         { ir_tick = 0; ADCSRA |= (1 << ADSC); }
14
15     /* Non-blocking I2C poll */
16     if (I2C_Slave_Poll(&rx_cmd, slave_status) == 1) {
17         switch (rx_cmd) {
18             case CMD_DISARMED: /* Reset all */ break;
19             case CMD_ARMED:     system_state = STATE_ARMED;
20                                 ir_armed = 1; break;
21             case CMD_ALARM:     system_state = STATE_ALARM; break;
22         }
23     }
24
25     /* Alarm check: local sensor OR master command */
26     uint8_t local_alarm = (dor_apnet || ir_trigger)
27                          && (system_state != STATE_DISARMED);
28     uint8_t show_alarm  = local_alarm || (system_state == STATE_ALARM);
29
30     if (show_alarm) {
31         OCR1A = SERVO_AAPEN; /* Servo blocks door */
32         slave_status = STATUS_DOOR_OPEN;
33
34         /* LED blink: 250ms (limit switch) eller 62ms (IR) */
35         uint16_t halvperiode = (local_alarm && dor_apnet) ? 250 : 62;
36         if ((uint16_t)(ms - last_blink) >= halvperiode) {
37             last_blink = ms;
38             led_state ^= 1;
39             if (led_state) PORTB |= (1 << PB0);
40             else          PORTB &= ~(1 << PB0);
41         }
42
43         /* Auto-reset etter 5s + 2s IR cooldown */
44         if (local_alarm && (uint16_t)(ms - alarm_start) >= 5000) {
45             dor_apnet = 0; ir_trigger = 0;
46             ir_cooldown = 2000; slave_status = STATUS_OK;
47         }
48     } else {
49         OCR1A = SERVO_LUKKET; /* Servo neutral */
50         PORTB &= ~(1 << PB0);
51     }
52 }
```

Listing 17: Timer0 COMP ISR – ADC-start og alarm-logikk (Slave1/main.c:58)

ADC startes automatisk hvert 40 ms (kun når armed og ikke i cooldown). Etter 5 sekunder tilbakestilles alarm-flaggene automatisk, og en 2-sekunders cooldown forhindrer umiddelbar re-trigger av IR-sensoren etter reset.

Når slave 1 oppdager innbrudd rapporteres dette via `slave_status`-variabelen som master leser via I²C MR-modus. Slave 1 svarer i ST-mode (statuskode 0xA8):

```
1 slave_status = STATUS_DOOR_OPEN; /* 0x01 -- alarm detected */
```

Listing 18: Slave status settes lokalt (Slave1/main.c)

6.3 Slave 2 – main.c

Slave 2 er reaksjons-noden og har en ISR som håndterer I²C, melodioppdatering og LED-blink:

```
1 ISR(TIMERO_COMP_vect)
2 {
3     static uint8_t siren_tick = 0;
4     static uint8_t system_state = STATE_DISARMED;
5     static uint8_t rx_cmd = 0;
6     static uint8_t led_state = 0;
7     static uint16_t last_blink = 0;
8
9     ms++;
10
11     /* Oppdater melodi hver 50ms nar buzzer aktiv */
12     if (buzzer_paa && ++siren_tick >= 50)
13         { siren_tick = 0; Buzzer_Melody_Update(); }
14
15     /* I2C poll */
16     if (I2C_Slave_Poll(&rx_cmd, STATUS_OK) == 1) {
17         switch (rx_cmd) {
18             case CMD_DISARMED:
19                 Buzzer_Off();
20                 PORTB &= ~(1 << PB0) | (1 << PB1);
21                 buzzer_paa = 0; break;
22             case CMD_ARMED:
23                 Buzzer_Off();
24                 PORTB = (PORTB & ~(1<<PB0)) | (1<<PB1); /* Yellow LED on */
25                 buzzer_paa = 0; break;
26             case CMD_ALARM:
27                 Buzzer_On();
28                 PORTB |= (1 << PB1);
29                 buzzer_paa = 1; break;
30         }
31     }
32
33     /* Red LED blink hver 250ms under alarm */
34     if (system_state == STATE_ALARM &&
35         (uint16_t)(ms - last_blink) >= 250)
36     {
37         last_blink = ms;
38         led_state ^= 1;
39         if (led_state) PORTB |= (1 << PB0);
40         else PORTB &= ~(1 << PB0);
41     }
42 }
```

Listing 19: Timer0 COMP ISR – Slave 2 (Slave2/main.c:29)

`Buzzer_Melody_Update()` i `buzzer.h` stegvis avspiller en meloditabell (D5–E5–G5–A5–Pause, gjenta) ved å oppdatere ICR1 og koble OC1A fra/til ved pause-noter.

6.4 I²C – i2c.h

`I2C_Master_Read()` leser én byte direkte fra slave i MR-modus. Slave svarer automatisk med sin `slave_status` uten at master sender noen request-byte:

```

1 uint8_t I2C_Master_Read(uint8_t slave_addr)
2 {
3     I2C_Start();
4     if (I2C_Status() != 0x08) { I2C_Stop(); return 0xFF; }
5
6     I2C_Write((slave_addr << 1) | 1); /* SLA+R */
7     if (I2C_Status() != 0x40) { I2C_Stop(); return 0xFF; }
8
9     uint8_t data = I2C_Read_NACK(); /* NACK = last byte */
10    I2C_Stop();
11    return data;
12 }

```

Listing 20: I2C_Master_Read – MR-modus (i2c.h)

Alle wait-loops har en timeout-teller (`I2C_TIMEOUT = 10000`) som bryter løkken ved kommunikasjonsfeil, sånn at systemet aldri henger i en evig loop.

7 Feilsøking og problemer underveis

7.1 LCD viste søppel tegn / ingenting

Symptom: LCD-bakgrunnslyset lyste, men ingen tegn viste seg. Ved noen forsøk vistes tilfeldige symboler.

Årsaker og løsninger:

1. **8-bit vs. 4-bit mismatch:** Første versjon av `lcd.h` var skrevet for 8-bit modus (`LCD_Data_Port = cmd`), men bare fire datapinner var koblet (D4–D7). LCD ignorerte kommandoene. Løsning: komplett omskriving til 4-bits nibble-overføring.
2. **Feil pin-antagelse:** Det ble antatt at datapinnene var PB4–PB7 (som i Assignment 6). Faktisk kobling ble avklart direkte: RS=PD0, E=PD3, D4–D7=PB0–PB3.
3. **Enable-timing:** For kort enable-puls (1 μ s) fungerte ikke pålitelig. Økt til 10 μ s high + 100 μ s lav ga stabil drift.

7.2 LCD startet på nytt under drift

Symptom: Etter noen minutter viste LCD “Smarthus Alarm / Starter opp...” igjen, som om MCU-en ble resettet.

Årsak: Spenningsfall (brownout) på VCC-skinnen når servo og LCD delte forsyning. Spenningsfallet under servo-bevegelse falt under MCU-ens reset-terskel og utløste en power-on-reset.

Løsning: 100 μ F elektrolytisk kondensator mellom VCC og GND nær ATmega32 for å buffer spenningsspikene.

7.3 I²C protokollkonflikt

Symptom: Alarm ble aldri rapportert til master. Slave 1 gikk tilbake til ARMED-tilstand i stedet for å sende alarm-status.

Årsak: `CMD_ARMED = 0x01` og `STATUS_DOOR_OPEN = 0x01` hadde identiske verdier. Den opprinnelige implementasjonen sendte en request-byte (0x01) til slave 1 via SLA+W (MT-modus) for å be om status. Slave tolket denne som `CMD_ARMED` og nullstilte alarm-flagget.

Løsning: Ny funksjon `I2C_Master_Read()` som gjør direkte MR-lesing (SLA+R) uten forutgående skriving. Slave svarer automatisk med `slave_status` ved ST-modus (status code 0xA8).

7.4 IR-sensor konstant trigger

Symptom: Alarm ble utløst kontinuerlig selv uten bevegelse foran sensoren.

Årsak: Threshold for lav – bordflater og vegger i labmiljøet lå innenfor deteksjonsavstand. IR-sensoren på gulvet reflekterte fra gulvflaten.

Løsning (iterativ kalibrering): Threshold ble gradvis økt: 200 → 300 → 330 → 500. Ved 500 (ca. 20 cm) ble triggering pålitelig. I tillegg ble det lagt til en 2-sekunders cooldown etter alarm-reset for å forhindre umiddelbar re-trigger:

```
1 ir_cooldown = 2000; /* 2 seconds -- forhindrer umiddelbar re-trigger */
```

Listing 21: IR cooldown etter alarm-reset (Slave1/main.c)

7.5 Servo-stalling

Symptom: Servoen snurret mot mekanisk stopp, viberte og ble varm.

Årsak: OCR1A=63 tilsvarer 504 μ s puls – under MG90S-ens minimum (500 μ s) – og presset servoen mot den fysiske grensen.

Løsning: Trygge verdier: SERVO_LUKKET = 188 ($\approx 1504 \mu$ s, nøytral) og SERVO_AAPEN = 300 ($\approx 2400 \mu$ s, dørblokker-posisjon).

7.6 Limit switch contact bounce

Symptom: En aktivering av vindussensoren ga 4–8 alarmblink i stedet for en sammenhengende alarm.

Årsak: Mekanisk contact bounce genererte flere korte INT0-interrupts ved ett trykk.

Løsning: 200 ms software debounce via `dor_timer` i ISR-en: ny INT0 ignoreres mens timeren er aktiv. Telleren dekrementeres hvert ms i Timer0 ISR:

```
1 if (dor_timer) dor_timer--;
```

Listing 22: Debounce-teller dekrementeres i Timer0 ISR (Slave1/main.c)

8 Diskusjon

8.1 Systemvirkemåte

Systemet fungerer som tiltenkt. Touch-sensoren veksler pålitelig mellom tilstandene, I²C-kommunikasjonen er stabil, og alarm utløses korrekt ved både vindussensor (limit switch) og IR-sensor (bevegelse på gulvet). Servoen blokkerer inngangsdøren effektivt ved alarm. Buzzeren på slave 2 gir tydelig lydvarsling, og LED-ene gir visuell bekreftelse.

Den inkrementelle testmetoden – en komponent om gangen, en slave om gangen – sparte mye feilsøkingstid. Da alle deler fungerte isolert, var I²C-integrasjonen den eneste ukjente faktoren.

8.2 Interrupt-drevet arkitektur

All tidsstyrt logikk kjøres i Timer0 COMP ISR (1 ms tick), og alle event-baserte funksjoner (touch, vindussensor, ADC complete) i dedikerte ISR-er. Dette gir forutsigbar responstid og eliminerer blokkerende `_delay_ms()`-kall i main-koden. Tomme while-løkker med `sleep_mode()` sikrer at CPU-en hviler mellom interrupts uten tap av funksjonalitet – alle nødvendige peripherals (Timer0, Timer1, TWI, ADC, INT0) fortsetter å kjøre i Idle mode.

8.3 Begrensninger og mulige forbedringer

- **Enkeltpoll:** Master poller kun slave 1. En mer robust implementasjon ville pollet begge slaver for å oppdage feil på slave 2.
- **Sikring av disarming:** Disarming via enkelt touch er lett å misbruke. En PIN-kode via gjentatte berøringer ville øke sikkerheten.
- **Strømforsyning:** Systemet kjøres fra USB. Med dedikert batteri og lavere klokkefrekvens ville Power-save mode gi lengre batterilevetid.
- **Utvidelse:** Systemet er enkelt å utvide med flere slaver (ulike adresser) uten å endre master-protokollen.

9 Konklusjon

Prosjektet endte med et fullt fungerende smarthus-alarmsystem montert i et 3D-printet to-etasjes hus. Systemet demonstrerer praktisk bruk av sentrale emner fra ING1507: digital I/O, external interrupts (INT0), Timer/Counter (CTC og Fast PWM), ADC med interrupt og I²C-kommunikasjon (master/slave).

Arbeidsfordelingen mellom de tre kortene – master for styring, slave 1 for sensorer og slave 2 for reaksjon – ga en ryddig struktur som var enkel å feilsøke og utvide. Feilsøkingprosessen ga verdifull praktisk forståelse av vanlige utfordringer: pin-misforståelser, protokollkonflikter, brownout og sensor threshold-kalibrering.

Referanser

Referanser

- [1] Marshad Kassim Mohamed, *Leksjoner i datamaskinarkitektur*, ING1507, Cyberingeniørskolen Jørstadmoen, 2026.
- [2] Microchip Technology, *ATmega32/L Datasheet*. [Online]. Tilgjengelig: <https://ww1.microchip.com/downloads/en/DeviceDoc/doc2503.pdf>
- [3] Sharp Microelectronics, *GP2Y0A21YK0F Distance Measuring Sensor Unit Datasheet*. https://global.sharp/products/device/lineup/data/pdf/datasheet/gp2y0a21yk_e.pdf.
- [4] *MG90S Micro Servo Motor Datasheet*. https://www.electronicoscaldas.com/datasheet/MG90S_Tower-Pro.pdf.
- [5] Hitachi, *HD44780U (LCD-II) Dot Matrix LCD Controller/Driver Datasheet*. <https://www.crystalfontz.com/controllers/datasheet-viewer.php?id=97>.
- [6] O. Hellebust-Nygaard og T. T. Haugland, *Smarthus – kildekode (GitHub)*. [Online]. Tilgjengelig: <https://github.com/Oskarhn/Smarthus>
- [7] O. Hellebust-Nygaard og T. T. Haugland, *Smarthus – video av systemet i drift (YouTube)*. [Online]. Tilgjengelig: <https://youtu.be/PLACEHOLDER>

A Vedlegg – Komplette kildekode